Abaixo está o conteúdo completo da Aula 3, pronto para salvar como atividade_aula_03_python.md.

Aula 3 — Simulação de ciclos de coleta de dados

Componente curricular

Linguagens de Programação e Banco de Dados

Curso

Curso Superior de Tecnologia em Automação Industrial

Instituição

Instituto Federal do Paraná — Campus Assis Chateaubriand

Unidade curricular

Unidade 3 — Estruturas de repetição aplicadas à coleta de dados industriais

Duração

4 horas-aula

---

1. Situação-problema

Nas aulas anteriores, o programa foi capaz de:

- Receber informações de um motor;
- Analisar temperatura, pressão e corrente;
- Classificar cada variável;
- Identificar situações de atenção, alarme e emergência;
- Recomendar uma ação ao operador.

Entretanto, o programa ainda analisa apenas uma leitura.

Em uma aplicação industrial, os sensores são lidos continuamente por um dispositivo de aquisição de dados. As leituras são realizadas em ciclos sucessivos:

Text Copiar Ciclo 1 → ler sensores Ciclo 2 → ler sensores Ciclo 3 → ler sensores Ciclo 4 → ler sensores ... Ciclo 20 → ler sensores

A equipe de automação precisa transformar o programa anterior em um **simulador de ciclos de coleta de dados industriais**.

O sistema deverá:

- Realizar várias leituras;
- Simular mudanças nos valores dos sensores;
- Analisar cada leitura;
- Identificar alarmes;
- Contabilizar ocorrências;
- Ignorar uma leitura quando houver falha de comunicação;
- Interromper a coleta em caso de emergência;
- Apresentar um resumo final da operação.

---

2. Objetivos da aula

Ao final da aula, o estudante deverá ser capaz de:

- Utilizar estruturas de repetição em Python;
- Aplicar os comandos `for` e `while`;
- Utilizar a função `range()`;
- Controlar a execução com `break`;
- Ignorar um ciclo específico com `continue`;
- Gerar valores simulados para sensores;
- Processar várias leituras automaticamente;
- Contabilizar alarmes e situações de emergência;
- Produzir um resumo dos ciclos realizados;
- Relacionar a repetição em Python com a varredura de um sistema industrial.

---

3. Produto final da aula

Ao final do encontro, cada estudante ou dupla deverá possuir um programa que:

1. Simule leituras de temperatura, pressão e corrente;
2. Execute vários ciclos de coleta;
3. Classifique cada leitura;
4. Exiba o estado do equipamento em cada ciclo;
5. Conte as situações normais, de atenção, de alarme e de emergência;
6. Simule uma falha de comunicação;
7. Interrompa a coleta em caso de emergência;
8. Apresente um resumo final da operação.

---

4. Conhecimentos utilizados

Esta aula utilizará conceitos trabalhados anteriormente:

- Variáveis;
- Tipos de dados;
- Operadores;
- Estruturas condicionais;
- Operadores lógicos;
- Classificação de estados;
- Relatórios com `print()`;
- Módulo `random`.

A principal novidade será a execução repetida de um conjunto de instruções.

---

5. Organização da aula

Etapa	Atividade	Tempo sugerido
1	Retomada da Aula 2 e apresentação do problema	20 min
2	Desafio 1 — repetindo uma mensagem	20 min
3	Estrutura `for` e função `range()`	25 min
4	Desafio 2 — criando ciclos de coleta	30 min
5	Simulação dos valores dos sensores	25 min
6	Desafio 3 — classificando cada ciclo	35 min
7	Intervalo ou pausa técnica	10 min
8	Desafio 4 — contando os estados	30 min
9	Desafio 5 — interrupção de emergência	25 min
10	Desafio 6 — falha de comunicação com `continue`	20 min
11	Testes, análise e fechamento	40 min
Total		**280 min**

> A distribuição pode ser ajustada conforme a duração adotada para cada hora-aula e o ritmo da turma.

---

6. Retomada da Aula 2

Abra o programa desenvolvido na aula anterior.

Na Aula 2, o programa recebia uma única leitura:

Text Copiar Temperatura: 85 °C Pressão: 6 bar Corrente: 12 A

E produzia uma classificação:

Text Copiar Estado geral: ALARME Ação recomendada: Verificar o equipamento e reduzir a carga.

Agora imagine que o dispositivo de coleta precise repetir essa operação:

Text Copiar Ler sensores Classificar valores Exibir estado Verificar alarmes Repetir

Perguntas para a turma

1. Como repetir um trecho de código várias vezes?
2. Seria necessário copiar o mesmo código 20 vezes?
3. Como controlar a quantidade de repetições?
4. Como interromper o programa antes do fim?
5. Como simular mudanças nos valores dos sensores?
6. O que deve acontecer se um sensor parar de responder?

Problema para projetar no quadro

Text Copiar Um dispositivo precisa realizar 20 ciclos de coleta.

Em cada ciclo, ele deve: - Ler três sensores; - Classificar os valores; - Exibir o estado do motor; - Contabilizar alarmes; - Parar se houver emergência.

---

7. Desafio 1 — Repetindo uma mensagem

Antes de simular sensores, vamos experimentar uma repetição simples.

Tarefa

Exiba cinco vezes a mensagem:

Text Copiar Iniciando ciclo de coleta...

Código

Python Copiar for ciclo in range(5): print(“Iniciando ciclo de coleta...”)

Observe que:

Python Copiar range(5)

produz os valores:

Text Copiar 0, 1, 2, 3, 4

Mostrando o número do ciclo

Python Copiar for ciclo in range(5): print(f”Iniciando ciclo {ciclo + 1} de coleta...”)

O resultado será semelhante a:

Text Copiar Iniciando ciclo 1 de coleta... Iniciando ciclo 2 de coleta... Iniciando ciclo 3 de coleta... Iniciando ciclo 4 de coleta... Iniciando ciclo 5 de coleta...

Testes rápidos

Altere a quantidade de ciclos para:

- 3;
- 5;
- 10;
- 20.

Perguntas rápidas

- Por que foi utilizado `ciclo + 1`?
- O que aconteceria se fosse utilizado apenas `ciclo`?
- O que o `range(10)` representa?
- Qual é a vantagem de usar um laço em vez de repetir o código manualmente?

---

8. Desafio 2 — Criando ciclos de coleta

Problema

Um dispositivo deve realizar dez ciclos de coleta e informar o número de cada ciclo.

Código inicial

Python Copiar quantidade_ciclos = 10

for ciclo in range(quantidade_ciclos): numero_ciclo = ciclo + 1

```
print()
print(f"========== CICLO {numero_ciclo} ==========")
print("Realizando leitura dos sensores...")
```

Tarefa

Adicione ao programa:

- Código do equipamento;

- Nome do dispositivo de coleta;
- Quantidade de ciclos;
- Mensagem de início da coleta;
- Mensagem de encerramento.

Exemplo

Python Copiar codigo_equipamento = “M-01” dispositivo_coleta = “CLP-SIM-01” quantidade_ciclos = 10

print(f”Equipamento: {codigo_equipamento}“) print(f”Dispositivo: {dispositivo_coleta}“)

for ciclo in range(quantidade_ciclos): numero_ciclo = ciclo + 1

```
print()
print(f"========== CICLO {numero_ciclo} ==========")
print("Realizando leitura dos sensores...")
```

print() print(“Coleta finalizada.”)

Perguntas rápidas

1. Qual trecho será executado várias vezes?
2. Qual trecho será executado apenas uma vez?
3. O que acontece com `numero_ciclo` em cada repetição?
4. O que aconteceria se `quantidade_ciclos` fosse igual a zero?

---

9. Simulação de valores de sensores

Para simular as leituras, será utilizado o módulo `random`.

Importação

No início do programa, adicione:

Python Copiar import random

Gerando uma temperatura inteira

Python Copiar temperatura = random.randint(60, 95)

Esse comando gera um número inteiro entre 60 e 95.

Gerando valores decimais

Python Copiar pressao = random.uniform(5.0, 10.0) corrente = random.uniform(10.0, 20.0)

O método `uniform()` gera um número decimal dentro do intervalo definido.

Teste isolado

Python Copiar import random

for ciclo in range(5): temperatura = random.randint(60, 95) pressao = random.uniform(5.0, 10.0) corrente = random.uniform(10.0, 20.0)

```
print()
print(f"Ciclo: {ciclo + 1}")
print(f"Temperatura: {temperatura:.2f} °C")
print(f"Pressão: {pressao:.2f} bar")
print(f"Corrente: {corrente:.2f} A")
```

Perguntas rápidas

- Por que os valores mudam a cada execução?
- O que acontece se alterarmos os limites de `randint()`?
- Qual é a diferença entre `randint()` e `uniform()`?
- Como gerar valores somente dentro de uma faixa normal?
- Como simular um sensor com comportamento mais previsível?

---

10. Desafio 3 — Classificando cada ciclo

Problema

A cada ciclo de coleta, o sistema deverá analisar os sensores e classificar o estado do motor.

Utilize os limites:

Python Copiar limite_temperatura = 80 limite_pressao = 8 limite_corrente = 15

Classificação da temperatura

Python Copiar if temperatura < 70: status_temperatura = “NORMAL” elif temperatura <= limite_temperatura: status_temperatura = “ATENÇÃO” else: status_temperatura = “ALARME”

Classificação da pressão

Python Copiar if pressao < 7: status_pressao = “NORMAL” elif pressao <= limite_pressao: status_pressao = “ATENÇÃO” else: status_pressao = “ALARME”

Classificação da corrente

Python Copiar if corrente < 13: status_corrente = “NORMAL” elif corrente <= limite_corrente: status_corrente = “ATENÇÃO” else: status_corrente = “ALARME”

Situações críticas

Python Copiar situacao_critica = (temperatura > 90 or pressao > 10 or corrente > 18)

Existência de alarme

Python Copiar existe_alarme = (status_temperatura == “ALARME” or status_pressao == “ALARME” or status_corrente == “ALARME”)

Existência de atenção

Python Copiar existe_atencao = (status_temperatura == “ATENÇÃO” or status_pressao == “ATENÇÃO” or status_corrente == “ATENÇÃO”)

Estado geral do ciclo

Python Copiar if situacao_critica: estado_geral = “EMERGÊNCIA” elif existe_alarme: estado_geral = “ALARME” elif existe_atencao: estado_geral = “ATENÇÃO” else: estado_geral = “NORMAL”

Exibição do ciclo

Python Copiar print() print(f”========== CICLO {numero_ciclo} ==========”) print(f”Temperatura: {temperatura:.2f} °C - {status_temperatura}”) print(f”Pressão: {pressao:.2f} bar - {status_pressao}”) print(f”Corrente: {corrente:.2f} A - {status_corrente}”) print(f”Estado geral: {estado_geral}”)

Teste orientado

Execute o programa algumas vezes e observe:

- Os valores são diferentes?
- O estado geral muda?
- O ciclo pode apresentar mais de uma variável em alarme?
- A situação de emergência encerra a coleta?

---

11. Desafio 4 — Contando os estados

Problema

Ao final da coleta, o supervisor deseja saber:

- Quantos ciclos foram realizados;
- Quantos ciclos ficaram normais;
- Quantos ciclos apresentaram atenção;
- Quantos ciclos apresentaram alarme;
- Quantos ciclos entraram em emergência.

Inicialização dos contadores

Antes do laço, crie:

Python Copiar ciclos_normais = 0 ciclos_atencao = 0 ciclos_alarme = 0 ciclos_emergencia = 0

Atualização dos contadores

Depois de determinar o `estado_geral`, acrescente:

Python Copiar if estado_geral == "NORMAL": ciclos_normais += 1 elif estado_geral == "ATENÇÃO": ciclos_atencao += 1 elif estado_geral == "ALARME": ciclos_alarme += 1 elif estado_geral == "EMERGÊNCIA": ciclos_emergencia += 1

Relatório final

Após o laço, acrescente:

Python Copiar print() print("==========") print(" RESUMO DA COLETA") print("==========") print(f"Total de ciclos planejados: {quantidade_ciclos}") print(f"Ciclos normais: {ciclos_normais}") print(f"Ciclos em atenção: {ciclos_atencao}") print(f"Ciclos em alarme: {ciclos_alarme}") print(f"Ciclos em emergência: {ciclos_emergencia}") print("==========")

Perguntas rápidas

- Por que os contadores começam com zero?
- Por que utilizamos `+= 1`?
- Em que momento cada contador deve ser atualizado?
- O que aconteceria se os contadores fossem criados dentro do laço?
- A soma dos contadores sempre será igual à quantidade planejada?

---

12. Desafio 5 — Interrupção em caso de emergência

Problema

Se o dispositivo identificar uma situação de emergência, o motor deverá ser parado imediatamente.

Utilize o comando `break`:

Python Copiar if estado_geral == “EMERGÊNCIA”: print(“PARADA DE EMERGÊNCIA ATIVADA.”) break

Aplicação

Coloque o trecho depois da exibição do estado geral:

Python Copiar print(f”Estado geral: {estado_geral}”)

if estado_geral == “EMERGÊNCIA”: print(“PARADA DE EMERGÊNCIA ATIVADA.”) break

Contando os ciclos realizados

Se a coleta for interrompida pelo `break`, o número de ciclos executados poderá ser menor que o número de ciclos planejados.

Antes do laço, crie:

Python Copiar ciclos_realizados = 0

Depois de confirmar que houve uma leitura válida, acrescente:

Python Copiar ciclos_realizados += 1

No relatório final:

Python Copiar print(f”Ciclos planejados: {quantidade_ciclos}”) print(f”Ciclos realizados: {ciclos_realizados}”)

Perguntas para discussão

- Por que uma emergência deve interromper o laço?
- Qual é a diferença entre ciclos planejados e ciclos realizados?
- O sistema deveria continuar coletando dados após uma emergência?
- Em uma aplicação real, além do `break`, que ação poderia ser executada?
- Como o evento de parada poderia ser registrado?

---

13. Desafio 6 — Ignorando uma leitura com `continue`

Problema

Em alguns momentos, o dispositivo pode estar em manutenção ou perder a comunicação com um sensor.

Quando isso acontecer, a leitura daquele ciclo deverá ser ignorada, mas a coleta deverá continuar no ciclo seguinte.

Simule uma falha de comunicação no ciclo 5:

Python Copiar if numero_ciclo == 5: print(“Falha de comunicação com o sensor.”) continue

Exemplo

Python Copiar for ciclo in range(10): numero_ciclo = ciclo + 1

```
print(f"Iniciando ciclo {numero_ciclo}")

if numero_ciclo == 5:
    print("Falha de comunicação. Leitura ignorada.")
    continue

print("Sensores lidos com sucesso.")
```

Contando leituras ignoradas

Antes do laço:

Python Copiar leituras_ignoradas = 0

Dentro do laço:

Python Copiar if numero_ciclo == 5: print(“Falha de comunicação com o sensor.”) leituras_ignoradas += 1 continue

No relatório:

Python Copiar print(f”Leituras ignoradas: {leituras_ignoradas}”)

Questões

1. O ciclo 5 será encerrado?
2. O laço completo será encerrado?
3. Qual é a diferença entre `continue` e `break`?
4. A leitura inválida deveria ser registrada em algum lugar?
5. O número de ciclos realizados deve incluir o ciclo com falha?

---

14. Programa completo de referência

Crie um arquivo chamado:

Text Copiar simulador_coleta.py

Utilize o código abaixo como referência para integrar os desafios:

Python Copiar import random

print(”==========”) print(” SIMULADOR DE COLETA INDUSTRIAL”) print(”==========”)

codigo_equipamento = “M-01” dispositivo_coleta = “CLP-SIM-01” quantidade_ciclos = 20

limite_temperatura = 80 limite_pressao = 8 limite_corrente = 15

ciclos_normais = 0 ciclos_atencao = 0 ciclos_alarme = 0 ciclos_emergencia = 0 ciclos_realizados = 0 leituras_ignoradas = 0

print(f”Equipamento: {codigo_equipamento}”) print(f”Dispositivo: {dispositivo_coleta}”) print(f”Ciclos planejados: {quantidade_ciclos}”)

for ciclo in range(quantidade_ciclos): numero_ciclo = ciclo + 1

```
print()
print(f"========== CICLO {numero_ciclo} ==========")

# Simulação de uma falha de comunicação
if numero_ciclo == 5:
    print("Falha de comunicação com o sensor.")
    leituras_ignoradas += 1
    continue

# Simulação das leituras
temperatura = random.randint(60, 95)
pressao = random.uniform(5.0, 10.0)
corrente = random.uniform(10.0, 20.0)

ciclos_realizados += 1

# Classificação da temperatura
if temperatura < 70:
    status_temperatura = "NORMAL"
elif temperatura <= limite_temperatura:
    status_temperatura = "ATENÇÃO"
else:
    status_temperatura = "ALARME"

# Classificação da pressão
if pressao < 7:
    status_pressao = "NORMAL"
elif pressao <= limite_pressao:
    status_pressao = "ATENÇÃO"
else:
    status_pressao = "ALARME"

# Classificação da corrente
if corrente < 13:
    status_corrente = "NORMAL"
elif corrente <= limite_corrente:
    status_corrente = "ATENÇÃO"
else:
    status_corrente = "ALARME"
```

```
# Identificação de situação crítica
situacao_critica = (
    temperatura > 90
    or pressao > 10
    or corrente > 18
)

# Identificação de alarme
existe_alarme = (
    status_temperatura == "ALARME"
    or status_pressao == "ALARME"
    or status_corrente == "ALARME"
)

# Identificação de atenção
existe_atencao = (
    status_temperatura == "ATENÇÃO"
    or status_pressao == "ATENÇÃO"
    or status_corrente == "ATENÇÃO"
)

# Classificação do estado geral
if situacao_critica:
    estado_geral = "EMERGÊNCIA"
elif existe_alarme:
    estado_geral = "ALARME"
elif existe_atencao:
    estado_geral = "ATENÇÃO"
else:
    estado_geral = "NORMAL"

# Contagem dos estados
if estado_geral == "NORMAL":
    ciclos_normais += 1
elif estado_geral == "ATENÇÃO":
    ciclos_atencao += 1
elif estado_geral == "ALARME":
    ciclos_alarme += 1
elif estado_geral == "EMERGÊNCIA":
    ciclos_emergencia += 1

# Exibição da leitura
print(f"Temperatura: {temperatura:.2f} °C - {status_temperatura}")
print(f"Pressão: {pressao:.2f} bar - {status_pressao}")
print(f"Corrente: {corrente:.2f} A - {status_corrente}")
print(f"Estado geral: {estado_geral}")

# Interrupção em caso de emergência
if estado_geral == "EMERGÊNCIA":
    print("PARADA DE EMERGÊNCIA ATIVADA.")
    break
```

print() print("==========") print(" RESUMO DA COLETA") print("==========") print(f"Equipamento: {codigo_equipamento}") print(f"Dispositivo: {dispositivo_coleta}") print(f"Ciclos planejados: {quantidade_ciclos}") print(f"Ciclos realizados: {ciclos_realizados}") print(f"Leituras ignoradas: {leituras_ignoradas}") print(f"Ciclos normais: {ciclos_normais}") print(f"Ciclos em atenção: {ciclos_atencao}") print(f"Ciclos em alarme: {ciclos_alarme}") print(f"Ciclos em emergência: {ciclos_emergencia}") print("==========")

---

15. Testes por cenários

Como os valores dos sensores são gerados aleatoriamente, os resultados podem mudar a cada execução.

Para testar situações específicas, substitua temporariamente os comandos de geração aleatória por valores fixos.

Cenário 1 — Operação normal

Python Copiar temperatura = 60 pressao = 5.0 corrente = 10.0

Resultado esperado:

Text Copiar Estado geral: NORMAL

Cenário 2 — Situação de atenção

Python Copiar temperatura = 75 pressao = 7.5 corrente = 14.0

Resultado esperado:

Text Copiar Estado geral: ATENÇÃO

Cenário 3 — Alarme de temperatura

Python Copiar temperatura = 85 pressao = 5.0 corrente = 10.0

Resultado esperado:

Text Copiar Estado geral: ALARME

Cenário 4 — Emergência por temperatura

Python Copiar temperatura = 95 pressao = 6.0 corrente = 12.0

Resultado esperado:

Text Copiar Estado geral: EMERGÊNCIA

A coleta deverá ser interrompida com:

Text Copiar PARADA DE EMERGÊNCIA ATIVADA.

Cenário 5 — Emergência por corrente

Python Copiar temperatura = 70 pressao = 6.0 corrente = 19.0

Resultado esperado:

Text Copiar Estado geral: EMERGÊNCIA

Cenário 6 — Múltiplos alarmes

Python Copiar temperatura = 92 pressao = 11.0 corrente = 19.0

Resultado esperado:

Text Copiar Estado geral: EMERGÊNCIA

---

16. Analisando os resultados

Execute o programa pelo menos três vezes e registre os resultados.

Execução	Ciclos planejados	Ciclos realizados	Leituras ignoradas	Normais	Atenção	Alarmes	Emergências
1							
2							
3							

Questões

1. Os resultados foram iguais em todas as execuções?
2. Por que os resultados mudaram?
3. O que influencia a quantidade de alarmes?
4. O que aconteceria se a faixa de geração da temperatura fosse alterada?
5. O que aconteceria se os limites fossem reduzidos?
6. O sistema sempre realiza todos os ciclos planejados?
7. Qual foi a causa da diferença entre ciclos planejados e realizados?
8. Qual é a diferença entre uma leitura ignorada e uma parada de emergência?
9. Em que situações o `continue` é mais adequado?
10. Em que situações o `break` é mais adequado?

---

17. Desafio de melhoria rápida

Escolha uma das melhorias abaixo.

Opção A — Permitir que o usuário informe a quantidade de ciclos

Substitua:

Python Copiar quantidade_ciclos = 20

por:

Python Copiar quantidade_ciclos = int(input(“Quantidade de ciclos de coleta:”))

Teste com:

- 3 ciclos;
- 5 ciclos;
- 10 ciclos.

Opção B — Permitir que o usuário defina a faixa de temperatura

Python Copiar temperatura_minima = int(input(“Temperatura mínima simulada:”))

temperatura_maxima = int(input(“Temperatura máxima simulada:”))

Depois, substitua:

Python Copiar temperatura = random.randint(60, 95)

por:

Python Copiar temperatura = random.randint(temperatura_minima, temperatura_maxima)

Teste diferentes faixas e observe a quantidade de alarmes.

Opção C — Contar alarmes individuais

Antes do laço:

Python Copiar total_alarmes = 0

Depois da classificação das variáveis:

Python Copiar if status_temperatura == “ALARME”: total_alarmes += 1

if status_pressao == “ALARME”: total_alarmes += 1

if status_corrente == “ALARME”: total_alarmes += 1

No resumo final:

Python Copiar print(f”Total de alarmes individuais: {total_alarmes}“)

Observe a diferença entre:

- Número de ciclos em alarme;
- Número total de variáveis em alarme.

Opção D — Criar um limite máximo de leituras

Mesmo que não ocorra emergência, a coleta deverá ser encerrada depois de determinado número de alarmes.

Antes do laço:

Python Copiar limite_de_alarmes = 3

Depois de atualizar os contadores:

Python Copiar if ciclos_alarme >= limite_de_alarmes: print(“Limite de alarmes atingido.”) break

Discuta:

- Por que essa regra poderia ser útil?
- Ela é igual a uma parada de emergência?
- Qual informação deveria ser registrada quando essa parada acontecer?

---

18. Entrega da atividade

Cada estudante ou dupla deverá entregar:

- Arquivo `simulador_coleta.py`;
- Programa funcionando com estrutura de repetição;
- Simulação de pelo menos três sensores;
- Classificação das leituras;
- Contagem dos estados;
- Uso de `break` em situação de emergência;
- Uso de `continue` para simular uma leitura ignorada;
- Registro dos testes realizados;
- Respostas às questões de análise;
- Uma melhoria adicional implementada.

---

19. Critérios de avaliação

Critério	Pontuação
Uso correto das estruturas de repetição	2,0
Simulação dos sensores	2,0
Classificação das leituras	2,0
Contagem dos estados e resumo final	1,5
Uso adequado de `break` e `continue`	1,0
Testes e interpretação dos resultados	1,0
Organização e clareza do código	0,5
Total	**10,0**

20. Fechamento da aula

Discuta com a turma:

1. Qual é a diferença entre `for` e `while`?
2. Quando o `for` parece mais adequado?
3. Quando o `while` pode ser mais adequado?
4. Qual é a função do `break`?
5. Qual é a função do `continue`?
6. Por que os valores simulados mudam a cada execução?
7. Como um sensor real poderia substituir o valor gerado pelo `random`?
8. Como as leituras poderiam ser armazenadas em uma lista?
9. Como registrar os ciclos em um arquivo?
10. Como enviar essas leituras para um banco de dados?
11. Qual informação deveria ser registrada quando ocorrer uma emergência?
12. O que seria necessário alterar para executar a coleta continuamente?

Registro de saída

Cada estudante deverá salvar:

- A versão final do programa;
- Os resultados dos testes;
- A melhoria implementada;
- Uma breve explicação sobre `for`, `break` e `continue`.

As perguntas finais preparam a próxima unidade da disciplina:

- Listas e históricos de leituras;
- Estruturas de dados;
- Estatísticas dos sensores;
- Organização dos registros coletados;
- Persistência dos dados em arquivos e, posteriormente, no PostgreSQL.